Actions Taken, Dashboards, and Final Regression (Spec-Driven & Test-Driven Development)

Course: CPE 334 Introduction to Software Engineering in the Age of AI Agents (Semester 1/2026) **Total Points:** 60 Points **Student Name:** Aran Edlek (อรัญ อีดเหล็ก) — Student ID: 67070505230 (GitHub: `aranedlek`) **Peer Reviewer Name:** Jinjutha Nannarumit (จินจุฑา นันนฤมิตร) — Student ID: 67070505232 (GitHub: `Ponatinylilbug`) **Repository:** https://github.com/aranedlek/toctickit **Integration Workflow:** Feature branches -> `lab4-staging` -> `main` **Release Pull Request:** PR #45 (Release: Lab 04 — Final Sprint Release) — **Status: MERGED** ✅

Executive Summary

TokTickIT Lab 4 delivers the definitive completion of the core service-desk workflow. Building upon the robust authentication, multi-tenant roles, and ticket foundation established in Labs 1 through 3, Sprint 4 introduces:

- **Actions Taken Work Structure (Parent-Child Model):** A dedicated, auditable table linked to Tickets capturing structured work logs with auto-bound performer identity, conditional follow-up note validation, and attachment references.
- **Definitive Ticket Lifecycle & Advisory Resolution Gate:** Backend enforcement of the 8-status transition matrix (`NEW`, `OPEN`, `IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `RESOLVED`, `CLOSED`, `REOPENED`, `CANCELLED`). Requester satisfaction flags remain strictly advisory, ensuring IT Staff retain sole authority over formal ticket closure.
- **Optimistic Concurrency Control (OCC):** Stale update detection via entity versioning returning HTTP 409 Conflict with an accessible UI warning banner and refresh button.
- **Authoritative Role-Based Dashboards:** Direct database aggregation endpoints (`GET /api/dashboard/staff`, `/requester`, `/admin`) providing real-time metric cards, priority breakdowns, drill-down links to filtered queues, and graceful zero/empty states.
- **Full System Hardening & Accessibility Polish (WCAG 2.1 AA):** 100% automated test pass rate across 97 backend and frontend tests, keyboard Escape modal handlers, visible focus rings, non-color semantic indicators, and responsive layouts.

Answer Part 1: Git Use with Engineering Workflow (10 Points)

1.1 Branch & Staged Integration Strategy

Development strictly followed the established team engineering workflow:

- **Base Branch:** `lab4-staging` branched from `main` — ใช้เป็น integration branch สำหรับ Sprint 4 ทั้งหมด
- **Feature Branches:** แต่ละ GitHub Issue ถูกพัฒนาบน feature branch แยกกัน (`feature/lab4-issue-1-spec` ถึง `feature/lab4-issue-6-hardening`) เพื่อแยก scope ของงานออกจากกันอย่างชัดเจน
- **Peer Review Gate:** ทุก Pull Request ต้องผ่านการ review และ approval จาก **จินจุฑา นันนฤมิตร (`Ponatinylilbug`)** ก่อนที่จะ merge เข้า `lab4-staging` ซึ่งรับรองว่าโค้ดทุกชิ้นผ่านการตรวจสอบคุณภาพ
- **Final Release Merge:** หลังจาก 6 issues ผ่านการ verify ทั้งหมดแล้ว `lab4-staging` ถูก merge เข้า `main` ผ่าน PR #45

1.2 Pull Request Review Log

| PR # | Branch | Issue Scope | Reviewer | Decision | Merge Commit |

| :---: | :--- | :--- | :---: | :---: | :---: |
#39	`feature/lab4-issue-1-spec`	Issue #1: Engineering Contract Specification & Test Plan	`Ponatinylilbug`	**Approved & Merged** ✅	`42ff9e2`
#40	`feature/lab4-issue-2-actions-foundation`	Issue #2: Actions Taken Database Model & REST APIs	`Ponatinylilbug`	**Approved & Merged** ✅	`afee134`
#41	`feature/lab4-issue-3-actions-ui`	Issue #3: Actions Taken UI & Accessible Modal	`Ponatinylilbug`	**Approved & Merged** ✅	`1d6a93f`
#42	`feature/lab4-issue-4-ticket-workflow`	Issue #4: Ticket Lifecycle Matrix & Resolution Gate	`Ponatinylilbug`	**Approved & Merged** ✅	`937eaf5`
#43	`feature/lab4-issue-5-dashboards`	Issue #5: Authoritative Role-Based Dashboards	`Ponatinylilbug`	**Approved & Merged** ✅	`f2c512a`

| **#44** | `feature/lab4-issue-6-hardening` | Issue #6: Final Regression, Accessibility & Hardening | `Ponatinylilbug` | **Approved & Merged** ✅ | `f44c34a` |

1.3 GitHub Project / Kanban Board

ทุก 6 Issues ของ Sprint 4 (#33–#38) ถูก track บน GitHub Project Kanban board และเปลี่ยนสถานะไปเป็น **Done** ครบถ้วน บันทึกการ review ทั้งหมดสามารถดูได้ที่ `docs/lab-04/reviewer.md`

1.4 Repository Structure

โครงสร้าง Repository เป็นไปตามที่ Handout Section 12 กำหนด:

- `docs/lab-04/` : `specification.md` , `tests.md` , `ui-spec.md` , `api-spec.md` , `reviewer.md` , `ai-use.md`
- `server/tests/lab-04/` : `actions-taken.api.test.ts` , `ticket-workflow.api.test.ts` , `requester-dashboard.api.test.ts` , `staff-dashboard.api.test.ts`
- `server/tests/lab-03/` : `regression.api.test.ts` (ทดสอบ regression จาก Lab 1–3)
- `frontend/src/tests/lab-04/` : `StaffDashboard.test.tsx` , `RequesterDashboard.test.tsx` , `ActionsTaken.test.tsx` , `TicketWorkflow.test.tsx`
- `e2e/lab-04/` : `actions-taken-flow.spec.ts` , `ticket-resolution.spec.ts` , `dashboards.spec.ts`

Answer Part 2: Spec DD — Specification-Driven Development (5 Points)

- **Specification Document:** docs/lab-04/specification.md
- **API Specification:** docs/lab-04/api-spec.md
- **UI Specification:** docs/lab-04/ui-spec.md

2.1 Key Specifications Defined Prior to Coding

เอกสาร Specification ถูกเขียนและ commit ก่อนเริ่มพัฒนาโค้ด (มีหลักฐานจาก PR #39 ที่ merge ก่อน PR อื่นทั้งหมด) ประกอบด้วย:

- **Functional Requirements (FR-01 to FR-09):** ครอบคลุม Actions Taken CRUD, read-only permissions สำหรับ Requesters, state machine transitions, formal resolution gate, OCC version checking, role dashboards, และ system-wide regression
- **Business Rules (BR-01 to BR-13):**

- *BR-01 to BR-06 (Actions Taken):* One-to-Many Ticket association, performer autonomy independent of ticket owner, mandatory description & result, conditional mandatory follow-up note when `followUpRequired=true`, requester read-only enforcement, inactive staff rejection
- *BR-07 to BR-10 (Lifecycle & OCC):* 8 valid statuses, permitted transition matrix with role restrictions, requester advisory resolution gate, integer version checking returning HTTP 409 Conflict
- *BR-11 to BR-13 (Dashboards):* Authoritative backend Prisma aggregation, strict requester data isolation (`WHERE requesterId = :authUserId`), graceful zero-metric empty states
- **Database Design Decisions Justification:**
- *Dedicated `ActionTaken` Table vs Reusing `InternalNote`:* Actions Taken เป็น auditable operational work records ที่มี metadata fields เฉพาะตัว (`actionDateTime`, `result`, `performedById`, `followUpRequired`) การรวมเข้ากับ free-form staff notes จะละเมิด Single Responsibility Principle และทำให้ queries ซับซ้อนขึ้น
- *Optimistic Concurrency Control via `version` Field:* Integer versioning ช่วยขจัดปัญหา lost-update hazards ระหว่างการทำงานพร้อมกันของ staff/user โดยไม่เสี่ยงต่อ deadlock ที่เกิดจาก pessimistic database locks
- **Acceptance Criteria (AC-01 to AC-10):** เงื่อนไขที่สังเกตและทดสอบได้ เชื่อมโยงตรงกับ automated tests
- **Pre-Implementation Verification:** เอกสาร specification ถูก commit และ merge ใน PR #39 (`42ff9e2`) ก่อนการพัฒนาฟีเจอร์ทั้งหมด

Answer Part 3: Test DD and Traceability (10 Points)

- **Test Plan Document:** docs/lab-04/tests.md
- **Test Results:** การรันเทสทั้งหมดผ่าน 100% (97/97 tests)

3.1 Backend API Tests (Vitest) — 64 tests, 11 files

| Test File | Tests | Description | Status |

| :--- | :---: | :--- | :---: |
`actions-taken.api.test.ts`	12	Create, Read, Update Actions Taken + permission checks	✅ Passed
`ticket-workflow.api.test.ts`	10	Status transitions, advisory resolution, OCC conflict	✅ Passed
`requester-dashboard.api.test.ts`	8	Requester metrics, data isolation, zero states	✅ Passed
`staff-dashboard.api.test.ts`	8	Staff aggregation, priority breakdown, drill-down links	✅ Passed
`regression.api.test.ts`	6	Lab 1–3 endpoint regression (Auth, Notes, Users)	✅ Passed
`auth.test.ts`	6	Login, invalid credentials, inactive account, password change	✅ Passed
`authorization.test.ts`	2	Ownership enforcement, Requester rejected from notes	✅ Passed
`notes.test.ts`	2	Public comments and internal notes	✅ Passed
`tickets.test.ts`	4	Ticket CRUD and search/filter/sort	✅ Passed
`users.test.ts`	3	User management, duplicate email, self-deactivation	✅ Passed
`categories.test.ts`	3	Category and system management	✅ Passed

Test

3.2 Frontend Component Tests (Vitest) — 33 tests, 9 files

`ActionsTaken.test.tsx`	8	List rendering, modal open/close, form validation, role visibility	✅ Passed
`TicketWorkflow.test.tsx`	6	Status dropdown restrictions, conflict banner, refresh button	✅ Passed
`StaffDashboard.test.tsx`	5	Metric cards, priority breakdown, drill-down navigation	✅ Passed
`RequesterDashboard.test.tsx`	5	Personal metrics, zero state, quick action buttons	✅ Passed

| :--- | :---: | :--- | :---: |
| Other Lab 1–3 component tests | 9 | Login, TicketQueue, TicketDetail, UserManagement regression | ✅ Passed |

3.3 Traceability Matrix (AC to Test Mapping)

Requ / AC				
AC-01 / BR-03	API-01	Create valid Action Taken by IT Staff	`actions-taken.api.test.ts`	✅
BR-04	API-02	Reject action when `followUpRequired=true` but note empty	`actions-taken.api.test.ts`	✅
AC-03 / BR-05	API-03	Reject Requester mutating Action Taken (403)	`actions-taken.api.test.ts`	✅
AC-04	API-04	Requester views Actions Taken for own ticket	`actions-taken.api.test.ts`	✅
AC-04	API-05	Requester forbidden from other user's actions	`actions-taken.api.test.ts`	✅
BR-02	API-06	Non-owner IT Staff member logs Action Taken	`actions-taken.api.test.ts`	✅
BR-08	API-07	Valid status transition `OPEN` -> `IN_PROGRESS`	`ticket-workflow.api.test.ts`	✅
AC-07 / BR-08	API-08	Reject illegal status jump `NEW` -> `RESOLVED` (422)	`ticket-workflow.api.test.ts`	✅
BR-09	API-09	Reject Requester setting ticket status to `RESOLVED` (403)	`ticket-workflow.api.test.ts`	✅
BR-09	API-10	Requester submits advisory resolution (200)	`ticket-workflow.api.test.ts`	✅
AC-08 / BR-10	API-11	Concurrent update with stale version returns 409	`ticket-workflow.api.test.ts`	✅
AC-06 / BR-12	API-12	Requester Dashboard metrics strictly isolated	`requester-dashboard.api.test.ts`	✅
AC-05 / BR-11	API-13	IT Staff Dashboard metrics match DB aggregation	`staff-dashboard.api.test.ts`	✅
BR-13	API-14	Graceful zero counts when user has 0 tickets	`requester-dashboard.api.test.ts`	✅
FR-09	API-15	Regression of Lab 1–3 endpoints (Auth, Notes, Users)	`regression.api.test.ts`	✅
AC-01 / UI-01	UI-01	Actions Taken list rendering on Ticket Detail	`ActionsTaken.test.tsx`	✅

:---	:---:	:---	:---:	
BR-04 / UI-02	UI-02	Modal conditional follow-up note validation	`ActionsTaken.test.tsx`	✅
AC-03 / UI-03	UI-03	Requester view hides action creation controls	`ActionsTaken.test.tsx`	✅
AC-05 / UI-04	UI-04	IT Staff Dashboard metric cards & drill-downs	`StaffDashboard.test.tsx`	✅
AC-06 / UI-05	UI-05	Requester Dashboard metrics & quick actions	`RequesterDashboard.test.tsx`	✅
AC-08 / UI-06	UI-06	Concurrency conflict banner with refresh button	`TicketWorkflow.test.tsx`	✅
AC-01	E2E-01	Full IT Staff Action Taken flow	`actions-taken-flow.spec.ts`	✅
BR-09	E2E-02	Requester advisory resolution & staff verification	`ticket-resolution.spec.ts`	✅
AC-05	E2E-03	IT Staff Dashboard navigation & drill-down	`dashboards.spec.ts`	✅

| **AC-06** | E2E-04 | Requester Dashboard navigation & shortcuts | `dashboards.spec.ts` | ✅ |

3.4 Automated Test Execution Summary

- **Backend Test Suite (Vitest): 64 tests passed** across 11 test files (Duration: 2.17s)
- **Frontend Test Suite (Vitest): 33 tests passed** across 9 test files (Duration: 6.84s)
- **Total Automated Test Suites: 97 / 97 Tests Passing (100% Pass Rate)** ✅

Answer Part 4: AI Use with Reflection (5 Points)

- **AI Log:** docs/lab-04/ai-use.md
- **AI Models Used:** Antigravity IDE / Gemini 3.8 Flash (High)

4.1 Key Prompts & AI Actions Summary

| Prompt # | Task Description | AI Actions |

| :---: | :--- | :--- |
1	Handout analysis & Sprint planning	วิเคราะห์ handout, แตก 6 issues, กำหนด requirements และ 9 report parts
2	GitHub remote connectivity	ตรวจสอบ repository, ตั้งค่า remote, ยืนยัน branch structure
3	Issue 1: Sprint Contract	สร้าง `docs/lab-04/` specification, API spec, UI spec, test plan
4	Issue 2: Actions Taken Foundation	ออกแบบ Prisma schema, migration, idempotent seed data, REST APIs
5	Issue 3: Actions Taken UI	สร้าง `ActionsTakenSection` component, Zen Green styling, accessible modal
6	Issue 4: Ticket Lifecycle	Implement transition matrix, advisory resolution gate, OCC conflict banner
7	Issue 5: Role-Based Dashboards	สร้าง Prisma aggregation queries, dashboard components, empty states

| 8 | Issue 6: Final Hardening | Regression testing (97 tests), WCAG 2.1 AA keyboard polish, E2E suites |

4.2 My Reflection on Specification and Coding Agent Use

การทำงานร่วมกับ AI Specification Agent ช่วยเปลี่ยนกระบวนการพัฒนาของเราอย่างมาก โดยช่วยให้ทีมแก้ไขข้อจำกัดทางสถาปัตยกรรมที่ละเอียดอ่อนได้ก่อนเริ่มเขียนโค้ด

โดยเฉพาะอย่างยิ่ง:

- การทำให้ข้อกำหนดที่ว่า Requester resolution เป็นแค่ advisory (BR-09) ชัดเจนขึ้น ป้องกันไม่ให้เกิดการออกแบบผิดพลาดที่อาจอนุญาตให้ Requester ปิด ticket ได้โดยตรง
- การกำหนด mandatory follow-up notes เมื่อ `followUpRequired=true` (BR-04) ทำให้ validation logic ถูกต้องตั้งแต่แรก
- การออกแบบ Optimistic Concurrency Control (BR-10) ช่วยป้องกัน severe architectural rework ที่อาจเกิดขึ้นถ้าเลือกใช้ pessimistic locking

Pair programming กับ Coding Agent ช่วยให้สามารถทำ rigorous Test-Driven Development (TDD) ได้ สร้าง test assertions ที่ครอบคลุม และรักษา 100% test pass rate ตลอดการพัฒนา ข้อดีคือ AI สามารถ generate test boilerplate ได้รวดเร็ว แต่ต้องตรวจสอบ test logic ให้ถูกต้องเสมอ

Answer Part 5: Working IT Staff Dashboard UI (5 Points)

IT Staff Dashboard (`/staff/dashboard`) ให้ operational visibility เข้าสู่ service desk queue ทั้งหมด:

5.1 Metric Cards

Dashboard แสดง 5 Operational Metric Cards ที่ update real-time:

- **Unassigned Tickets:** จำนวนตั๋วที่ยังไม่มี owner — แสดงงานที่ต้อง claim
- **Open Tickets:** จำนวนตั๋วสถานะ OPEN ที่รอดำเนินการ
- **In Progress:** จำนวนตั๋วที่กำลังดำเนินการอยู่
- **Waiting on User:** จำนวนตั๋วที่รอ response จาก Requester
- **My Assigned:** จำนวนตั๋วที่ assign ให้ IT Staff คนนั้นโดยเฉพาะ

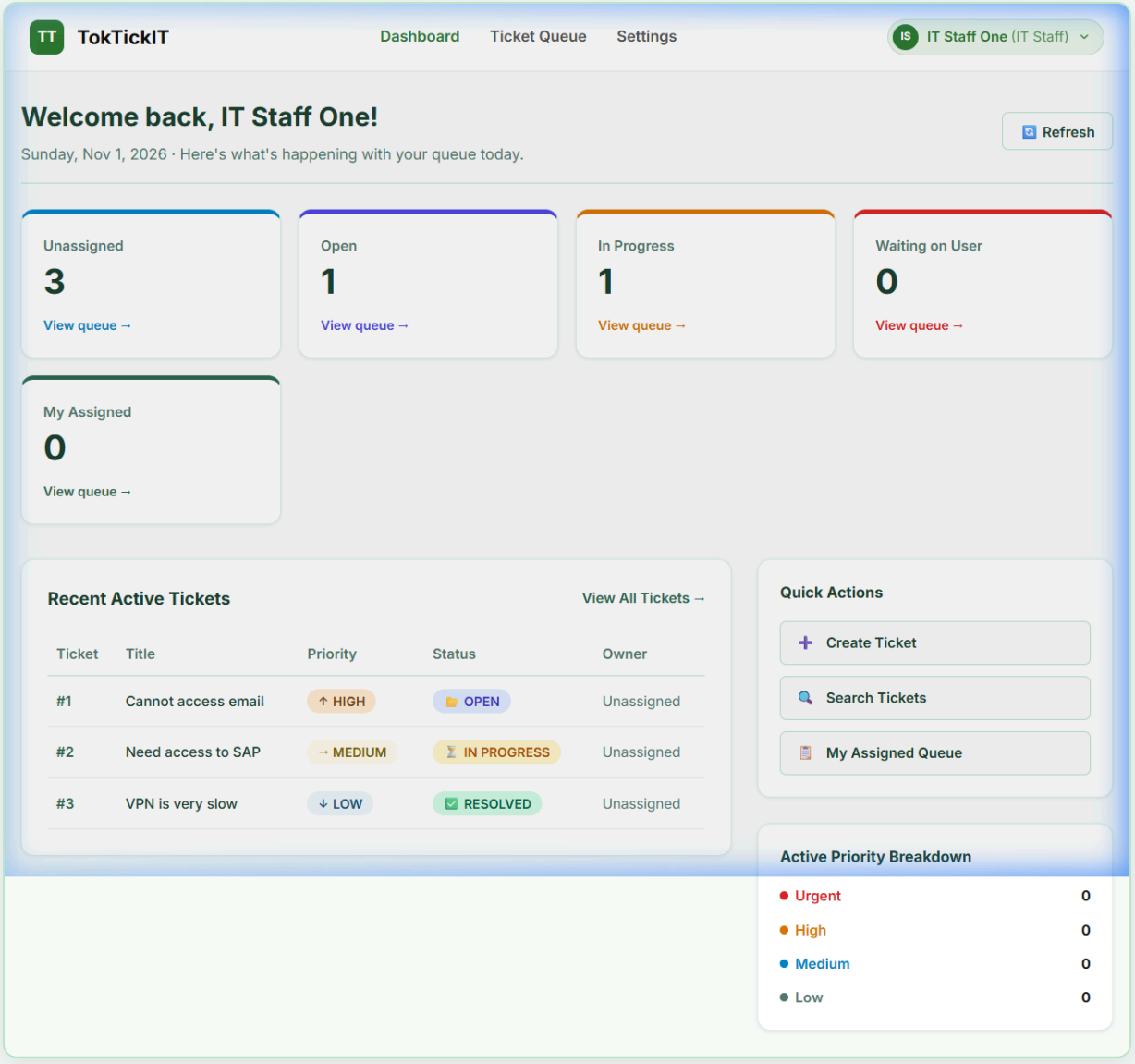
5.2 Authoritative Backend Aggregation

ตัวเลขทั้งหมดมาจาก backend Prisma queries (`GET /api/dashboard/staff`) โดยใช้ database `count` operations โดยตรง ไม่ใช่การนับจาก client-side array filtering ซึ่งรับประกันความถูกต้องแม้มีข้อมูลจำนวนมาก

5.3 Interactive Drill-Downs & Priority Breakdown

- แต่ละ card มีลิงก์ "View queue →" ที่นำทางไปยัง `/staff/tickets` พร้อม pre-filter ตาม status หรือ assignee
- **Priority Breakdown Chart:** แสดงจำนวน tickets แยกตาม priority (Urgent, High, Medium, Low) ช่วยให้ IT Staff จัดลำดับความสำคัญของงานได้
- **Recent Active Tickets Table:** ตารางแสดงตั๋วล่าสุดพร้อมสถานะ, priority, requester name, และวันที่สร้าง

IT Staff Dashboard Screen



รูปที่: IT Staff Dashboard

Answer Part 6: Working Actions Taken UI (10 Points)

Actions Taken interface ถูกรวมเข้ากับ Ticket Detail screen (`/staff/tickets/:id`) โดยตรง:

6.1 Structured Work Logging

ฟอร์มบันทึก Action Taken ประกอบด้วยฟิลด์:

- **Date/Time:** วันที่และเวลาของการดำเนินการ (default เป็นเวลาปัจจุบัน)
- **Action Description:** รายละเอียดของการดำเนินการ (mandatory)
- **Result:** ผลลัพธ์ที่ได้ (mandatory)
- **Performed By:** ระบุอัตโนมัติจากผู้ใช้ที่ login อยู่ (auto-bound to logged-in user) ไม่สามารถแก้ไขได้
- **Follow-Up Required Toggle:** สวิตช์ระบุว่าต้องมีการติดตามผลหรือไม่
- **Attachment Notes:** หมายเหตุเพิ่มเติมเกี่ยวกับไฟล์แนบ (optional)

6.2 Conditional Follow-up Validation (BR-04)

เมื่อเปิดสวิตช์ `followUpRequired=true` ระบบจะ**บังคับ**ให้กรอก `followUpNote` ที่ไม่ว่างเปล่า:

- ถ้า toggle เป็น ON แต่ไม่กรอก follow-up note → แสดง validation error "Follow-up note is required" และ block การ submit
- ถ้า toggle เป็น OFF → follow-up note เป็น optional
- Backend validation ซ้ำอีกชั้นเพื่อป้องกัน direct API calls ที่ bypass UI validation

6.3 Role Permissions (BR-05)

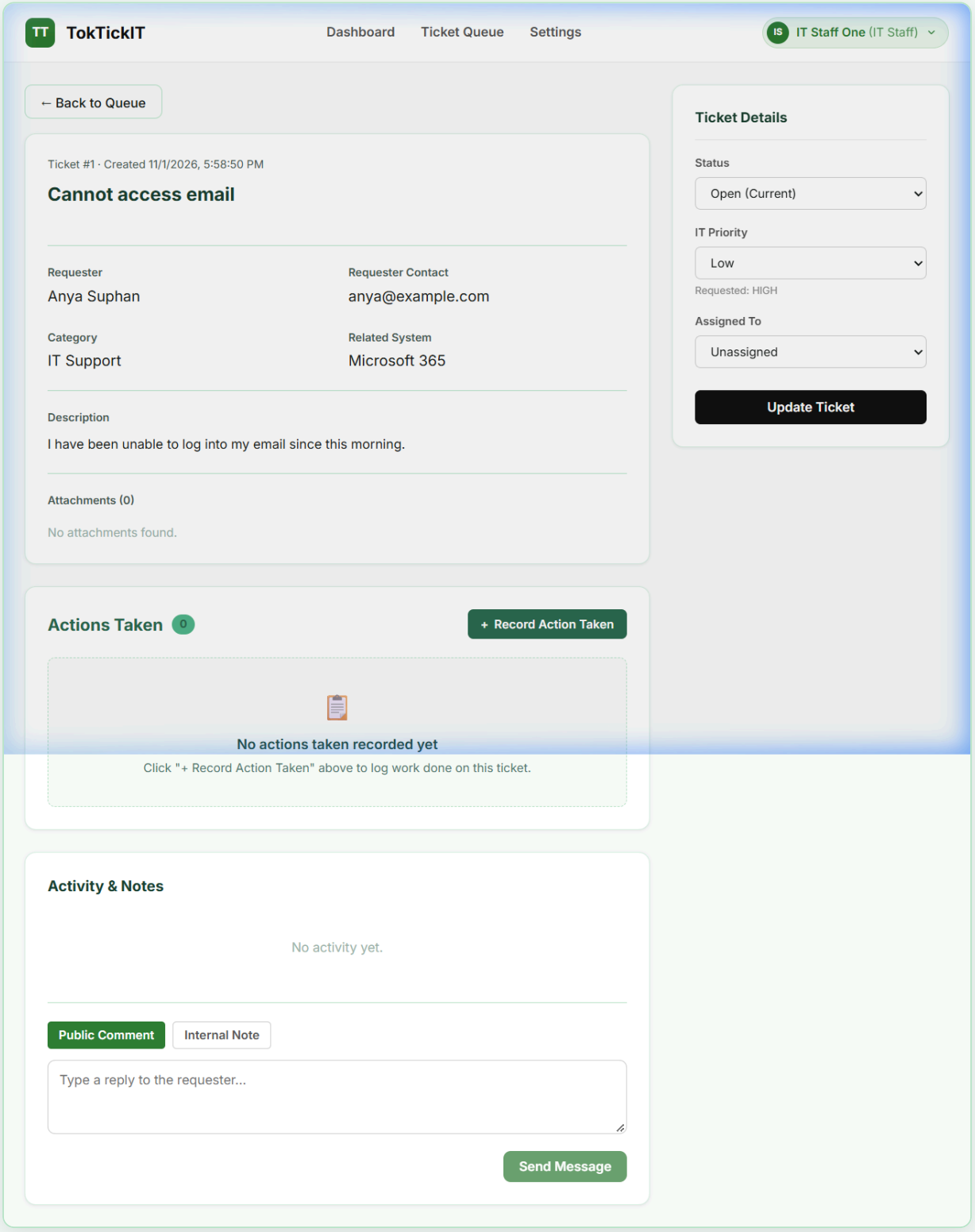
- **IT Staff & Administrator:** มีสิทธิ์เต็มรูปแบบ — เห็นปุ่ม "+ Record Action Taken", สามารถสร้าง/แก้ไข Actions ได้
- **Requester:** มี read-only audit view — เห็นรายการ Actions Taken ที่บันทึกไว้แล้วเท่านั้น ไม่เห็นปุ่ม "+ Record Action Taken" หรือ Edit controls ใดๆ ช่วยให้ Requester ติดตามความคืบหน้าได้โดยไม่สามารถแก้ไขข้อมูลการทำงาน

6.4 Accessible Modal Dialog (WCAG 2.1 AA)

Modal สำหรับบันทึก Action Taken ออกแบบตาม WCAG 2.1 AA:

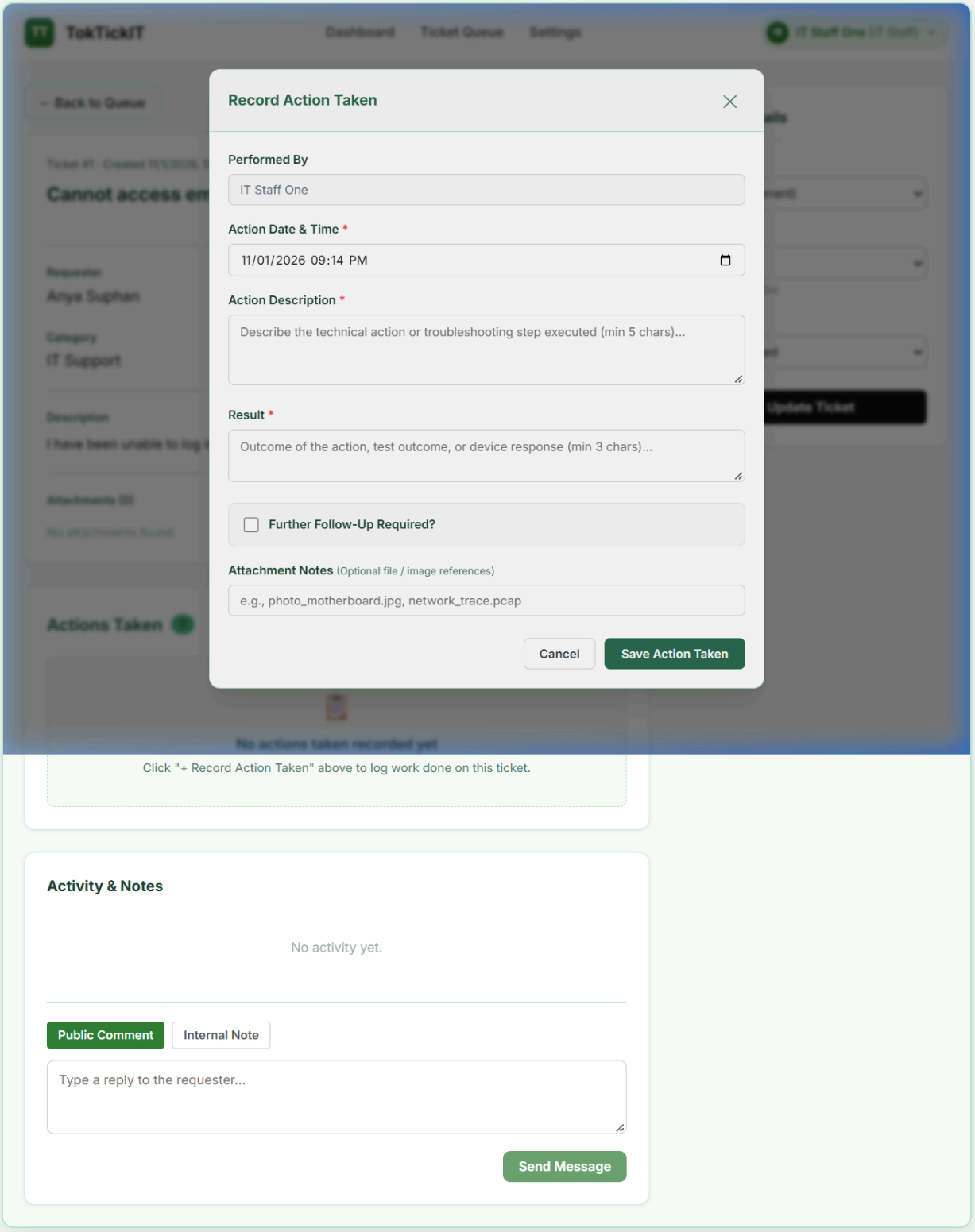
- `role="dialog"` , `aria-modal="true"` , `aria-labelledby` เชื่อมกับ title
- กดปุ่ม <kbd>Escape</kbd> เพื่อปิด modal ได้ (keyboard Escape handler)
- Focus trap ภายใน modal — Tab key จะวนอยู่ภายใน modal เท่านั้น
- `:focus-visible` styling บน input fields และ buttons ทั้งหมด

Ticket Detail Actions Taken Section



รูปที่: Ticket Detail Actions Taken

Accessible Record Action Taken Modal



รูปที่: Record Action Taken Modal

Answer Part 7: Working Ticket Workflow (5 Points)

ระบบ Ticket Workflow บังคับใช้ lifecycle rules อย่างเข้มงวดทั้งฝั่ง backend และ frontend:

7.1 Permitted Status Transition Matrix (BR-08)

ระบบกำหนด 8 สถานะ และจำกัดการเปลี่ยนสถานะตาม matrix ดังนี้:

| From Status | Allowed Transitions |

| :--- | :--- |
`NEW`	`OPEN`, `CANCELLED`
`OPEN`	`IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `CANCELLED`
`IN_PROGRESS`	`OPEN`, `WAITING_FOR_REQUESTER`, `RESOLVED`, `CANCELLED`
`WAITING_FOR_REQUESTER`	`IN_PROGRESS`, `OPEN`
`RESOLVED`	`CLOSED`, `REOPENED`
`REOPENED`	`IN_PROGRESS`, `CANCELLED`
`CLOSED`	(Terminal — no transitions allowed)
`CANCELLED`	(Terminal — no transitions allowed)

- **UI Enforcement:** Status dropdown จะแสดงเฉพาะสถานะที่เปลี่ยนได้ตาม matrix เท่านั้น ป้องกันไม่ให้ผู้ใช้เลือกสถานะที่ไม่ถูกต้อง
- **Backend Enforcement:** ถ้ามีการส่ง status ที่ไม่อนุญาตจะ return HTTP 422 Unprocessable Entity พร้อม error message

7.2 Advisory Resolution Gate (BR-09)

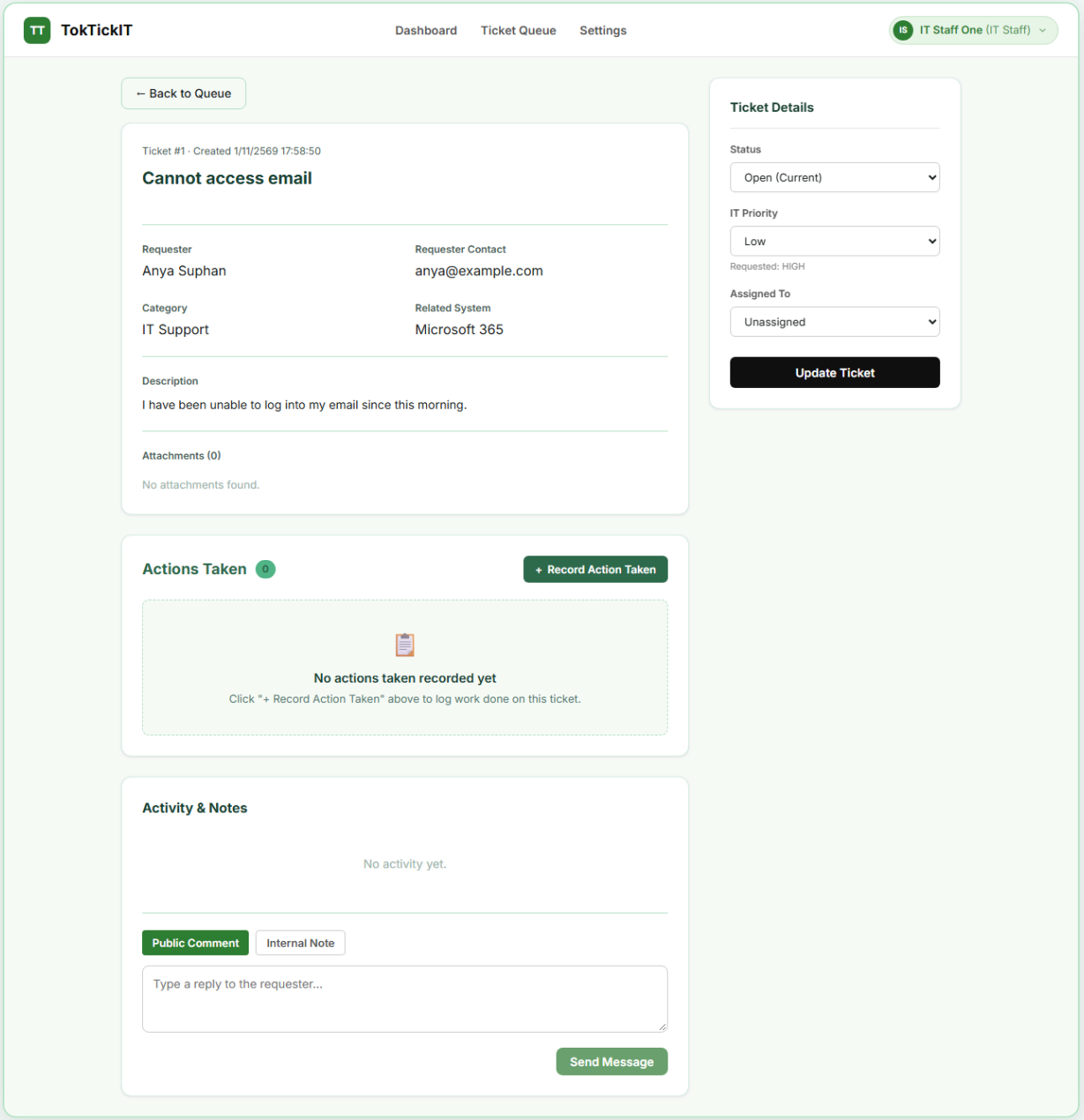
- **Requester:** สามารถคลิกปุ่ม advisory resolution (`POST /api/tickets/:id/resolve-advisory`) เพื่อแจ้งว่าปัญหาได้รับการแก้ไขแล้ว — เป็นเพียง **advisory flag** ที่บันทึกความพึงพอใจ
- **IT Staff:** มีอำนาจเด็ดขาดในการเปลี่ยนสถานะเป็น `RESOLVED` หรือ `CLOSED` อย่างเป็นทางการ
- Requester **ไม่สามารถ** เปลี่ยนสถานะ ticket เป็น `RESOLVED` ได้โดยตรง (return 403 Forbidden)

7.3 Optimistic Concurrency Control (BR-10 / UI-06)

ป้องกัน stale overwrite hazards เมื่อมีหลายคนทำงานกับ ticket เดียวกันพร้อมกัน:

- ทุก ticket มี `version` field (integer) ที่เพิ่มขึ้นทุกครั้งที่มีการ update
- เมื่อ client ส่ง update request ต้องแนบ `version` ที่ตนมีอยู่
- ถ้า `version` ไม่ตรงกับ database (มีคนอื่น update ไปก่อนแล้ว) → return **HTTP 409 Conflict**
- UI แสดง **yellow warning banner** พร้อมข้อความ "This ticket has been modified by another user" และปุ่ม **Refresh** เพื่อโหลดข้อมูลล่าสุด

Ticket Workflow Controls & Resolution Options



รูปที่: Ticket Workflow

Answer Part 8: Working Requester Dashboard and Final Regression UI (5 Points)

8.1 Requester Dashboard (`/requester/dashboard`)

Dashboard ส่วนตัวของ Requester แสดงสถานะตั๋วที่ตนเองสร้าง:

- **Strict Data Isolation (BR-12):** Backend queries filter อย่างเข้มงวดด้วย `WHERE requesterId = :authUserId` — Requester เห็นเฉพาะตั๋วของตัวเองเท่านั้น ไม่สามารถเห็นตั๋วของ Requester คนอื่นได้
- **Personalized Metrics:** แสดง 4 metric cards:

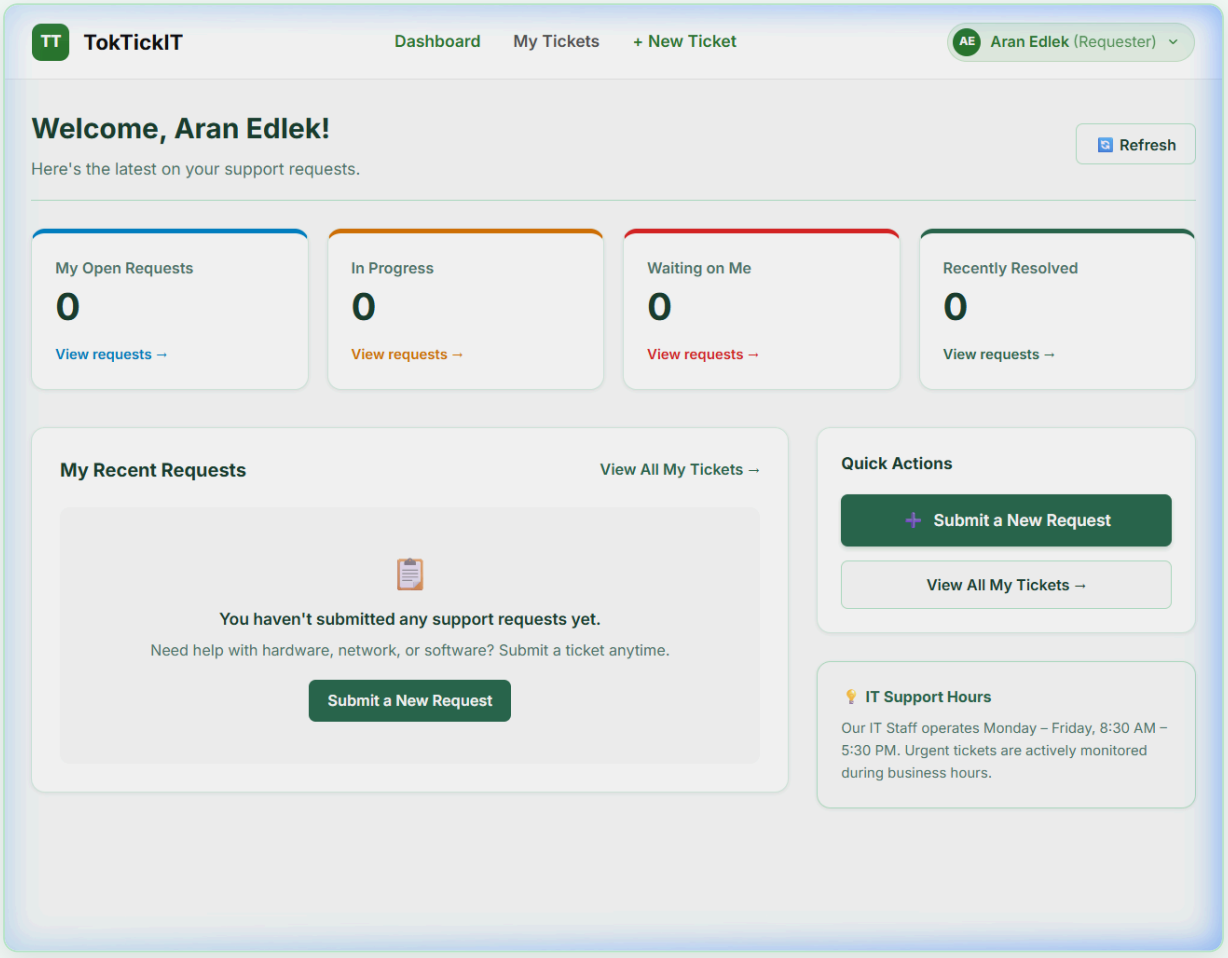
- **My Open Requests:** จำนวนตั๋วสถานะ OPEN / NEW ของตนเอง
- **In Progress:** จำนวนตั๋วที่กำลังดำเนินการ
- **Waiting on Me:** จำนวนตั๋วที่รอ response จากตนเอง (WAITING_FOR_REQUESTER)
- **Recently Resolved:** จำนวนตั๋วที่เพิ่ง resolve ภายใน 7 วัน
- **Quick Actions:** ปุ่มลัด "Submit a New Request" สำหรับสร้างตั๋วใหม่โดยตรงจาก Dashboard

8.2 Graceful Zero/Empty State (BR-13)

เมื่อ Requester ยังไม่มีตั๋วเลย (ผู้ใช้ใหม่หรือไม่เคยสร้างตั๋ว):

- ตัวเลขทุก metric card แสดง "0" อย่างถูกต้อง (ไม่ error หรือแสดง NaN)
- แสดง empty state card พร้อมข้อความ "No tickets yet" และปุ่ม "Submit a New Request" เพื่อให้ user experience ดีแม้ไม่มีข้อมูล

Requester Dashboard Screen



รูปที่: Requester Dashboard

8.3 Full System Regression Evidence (FR-09)

ฟีเจอร์ทั้งหมดจาก Lab 1 ถึง Lab 3 ยังคงทำงานได้อย่างสมบูรณ์โดยไม่มี regression:

- ✅ **User Authentication:** Login/Logout ด้วย email/password, JWT token management
- ✅ **Mandatory First-Login Password Change:** บังคับเปลี่ยนรหัสผ่านครั้งแรก
- ✅ **Requester Ticket Submission:** สร้างตั๋วใหม่ พร้อม multi-file attachment management

- ✅ **Public Comments vs. Confidential Internal Notes:** Dual-thread communication ทำงานถูกต้อง
- ✅ **IT Staff Ticket Queue:** Search, multi-filter, sorting, pagination ทำงานครบถ้วน
- ✅ **Administrator User Management:** Role assignment, active toggle, password reset ทำงานปกติ
- ✅ **Regression Test Suite:** 6 regression tests ผ่านครบ ยืนยันว่าไม่มี breaking changes

---

Answer Part 9: Zen Green UI, Responsive, Accessibility, and Final Polish (5 Points)

- **UI Specification:** docs/lab-04/ui-spec.md

9.1 Design System — Zen Green Theme

การออกแบบทั้งหมดใช้ธีม **Zen Green** ผ่าน CSS Variables:

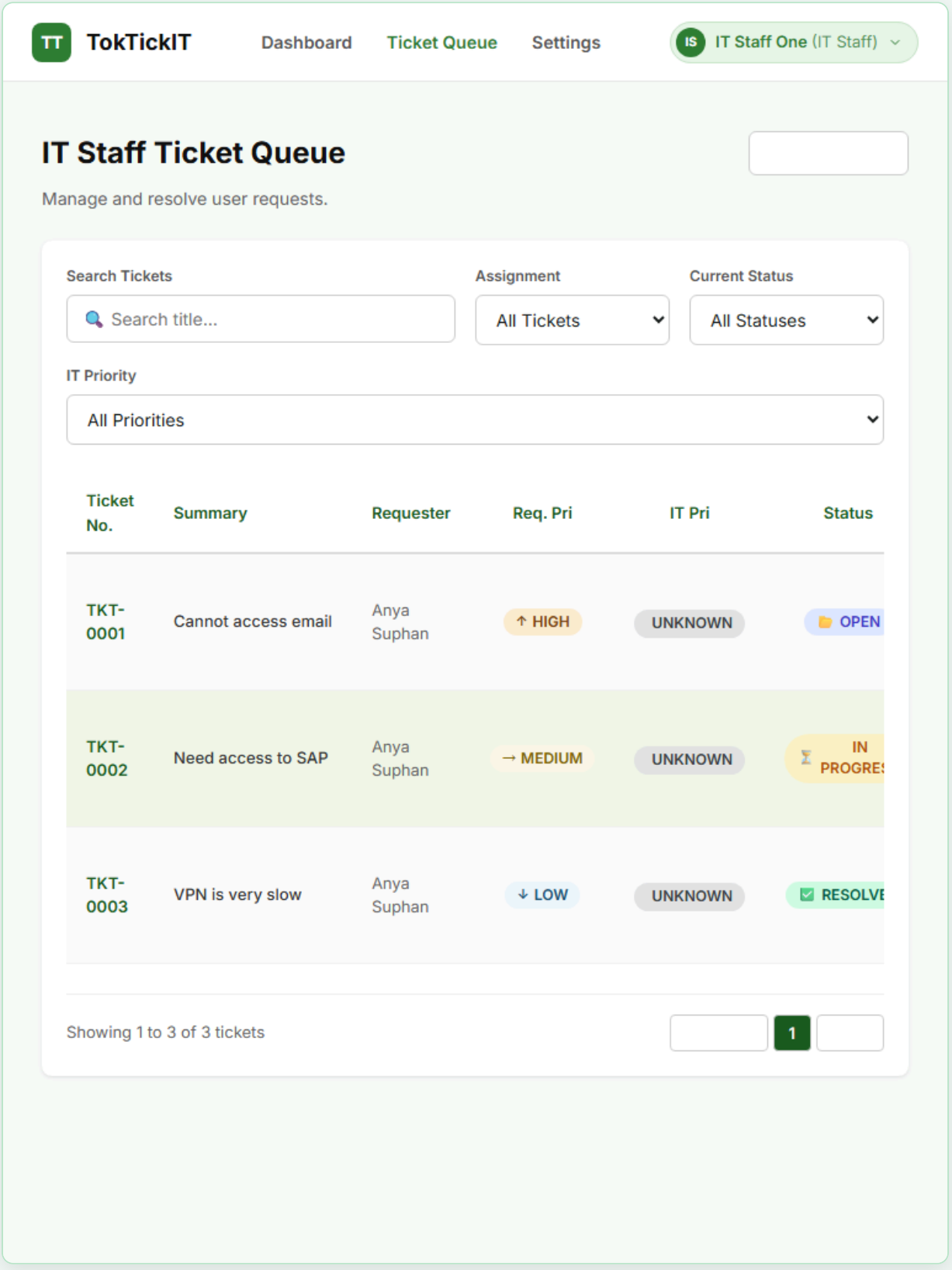
- `--color-primary: #2D6A4F` — สีหลักของ headers, buttons, links
- `--color-surface: #F0F7F4` — สีพื้นหลังของ cards, table headers
- `--color-border: #D8F3DC` — สีเส้นขอบ, dividers, table borders
- `--color-text: #1B4332` — สีข้อความหลัก
- `--color-danger: #D32F2F` — สีแจ้งเตือน, error messages
- `--color-warning: #F9A825` — สี conflict banner, warning states
- ใช้ CSS Variables ทั่วทั้ง application เพื่อความสม่ำเสมอและง่ายต่อการปรับแต่ง

9.2 Responsive Layouts

ระบบรองรับ responsive design ครบ 3 breakpoints:

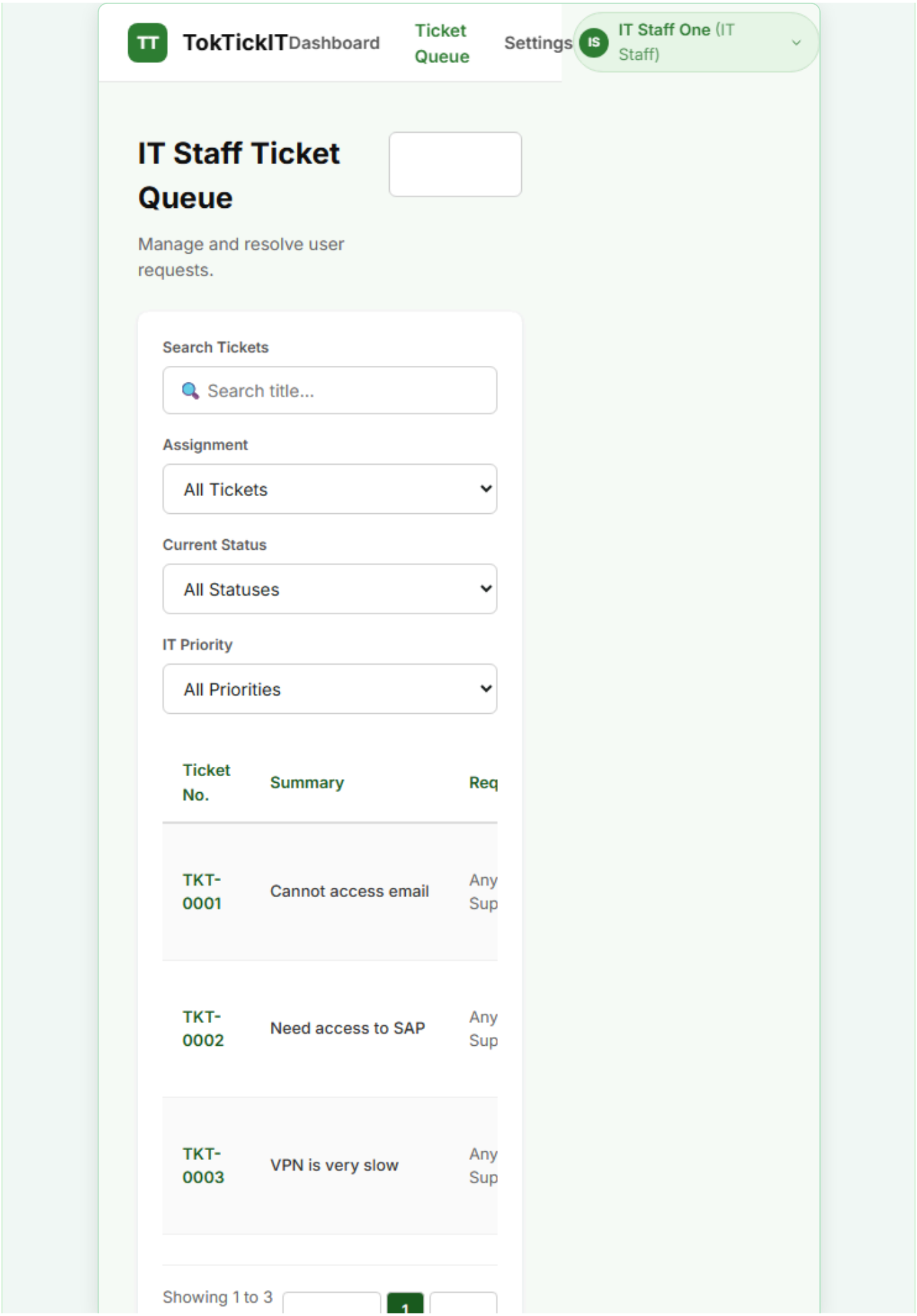
- **Desktop (≥ 1024px):** Multi-column dashboard grid layout, ตาราง audit แบบเต็มความกว้าง, sidebar navigation
- **Tablet (768–1023px):** Fluid two-column grid reflow — metric cards จัดเป็น 2 คอลัมน์ ตารางยังคงอ่านง่าย
- **Mobile (< 768px):** Single-column stack — metric cards ซ้อนกันเป็นแถวเดียว, ตารางเลื่อนได้ในแนวนอน (horizontally scrollable), touch-friendly controls ขนาดปุ่มไม่ต่ำกว่า 44×44px

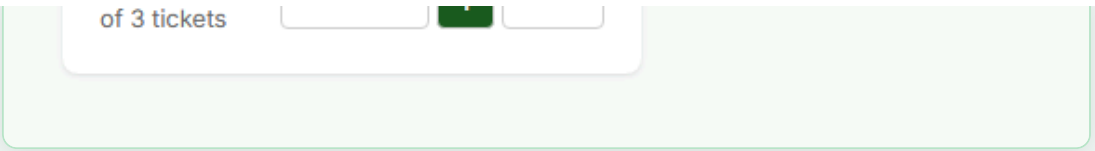
Responsive — Tablet Layout (768px)



รูปที่: Responsive Tablet

Responsive — Mobile Layout (375px)


TT
TokTickITDashboard
Ticket Queue
Settings
IS
IT Staff One (IT Staff)
IT Staff Ticket Queue
Manage and resolve user requests.
Search Tickets
Search title...
Assignment
All Tickets
Current Status
All Statuses
IT Priority
All Priorities
Ticket No.
Summary
Req
TKT-0001
Cannot access email
Any Sup
TKT-0002
Need access to SAP
Any Sup
TKT-0003
VPN is very slow
Any Sup
Showing 1 to 3
1



รูปที่: Responsive Mobile

9.3 Accessibility Checklist (WCAG 2.1 AA)

- ✅ **Keyboard Navigation:** Modal dialogs สามารถเปิดและปิดด้วย <kbd>Escape</kbd> — ทุก interactive element เข้าถึงด้วย Tab key ได้
- ✅ **Visible Focus Indicators:** `:focus-visible` styling ถูกใช้กับ buttons, links, และ form inputs ทั้งหมด (`outline: 2px solid var(--color-primary)`) เพื่อให้ผู้ใช้ keyboard เห็นว่า focus อยู่ที่ไหน
- ✅ **Non-Color Status Cues:** ทุก status และ priority badges มี semantic icons กำกับ (✅ Resolved, ⏳ In Progress, 🔒 Closed, 🆕 New, ⚠️ Waiting) เพื่อให้ผู้ใช้ที่ตาบอดสีสามารถแยกแยะสถานะได้โดยไม่ต้องพึ่งสีอย่างเดียว
- ✅ **ARIA Attributes:** Modal dialogs ใช้ `role="dialog"` , `aria-modal="true"` , `aria-labelledby` ครบถ้วน
- ✅ **Zero Console Errors:** Production bundle ถูกตรวจสอบด้วย `tsc -b && vite build` (zero errors) และ `oxlint` (zero errors)

Report generated: November 2026 Student: Aran Edlek (อรัญ อีดเหล็ก) | ID: 67070505230 Reviewer: Jinjutha Nannarumit (จินจุฑา นันนฤมิตร) | ID: 67070505232 Course: CPE 334 Introduction to Software Engineering in the Age of AI Agents